

Aufgabe 1 *Assembly und Pseudoinstruktionen*

1.1. Grenzen Sie Instruktionen und Pseudoinstruktionen voneinander ab. Welche Funktion haben Segmente in einem Assembler-Programm?

Instruktionen sind native Befehle, die direkt von der Hardware ausgeführt werden können (es gibt eine 1:1-Entsprechung zum Maschinencode).

Pseudoinstruktionen sind Hilfskonstrukte für den Programmierer, die es in der Hardwarearchitektur physisch nicht gibt. Sie werden vom Assembler automatisch in eine oder mehrere echte Maschineninstruktionen übersetzt (z. B. wird `li` zu `addi`).

Funktion der Segmente: Segmente strukturieren den Speicher des Programms:

- `.text`: Enthält den eigentlichen, ausführbaren Programmcode (Maschinenbefehle).
- `.data`: Enthält statische Daten (wie globale Variablen, Konstanten oder Arrays), die zu Programmbeginn im Arbeitsspeicher initialisiert werden.

1.2. Identifizieren Sie alle Pseudoinstruktionen des Programmes und geben Sie an, in welche Instruktionen sie aufgeschlüsselt werden. Erklären Sie, warum die Pseudoinstruktionen so übersetzt werden.

1 .data	
2 zahl: .word 0x09fa00ce	
3	
4 .text	
5 beginning:	
6 lh t0 zahl	00000000 <beginning>: 0: 10000297 auipc x5 0x10000
7 srli t0 t0 4	4: 00029283 lh x5 0 x5
8 li t1 0	8: 0042d293 srli x5 x5 4
9 middle:	c: 00000313 addi x6 x0 0
10 add t2 t2 t0	00000010 <middle>: 10: 005383b3 add x7 x7 x5
11 addi t0 t0 -1	14: fff28293 addi x5 x5 -1
12 bne t0 t1 middle	18: fe629ce3 bne x5 x6 -8 <middle>
13 end:	0000001c <end>: 1c: 00000013 addi x0 x0 0
14 nop	

Abbildung 1: Auflösung der Pseudoinstruktionen in Ripes

Die identifizierten Pseudoinstruktionen sind:

1. `lh t0, zahl`

- *Übersetzung:* `auipc x5, 0x10000` gefolgt von `lh x5, 0(x5)`
- *Erklärung:* RISC-V-Instruktionen haben eine feste Länge von 32 Bit. Eine vollständige 32-Bit-Speicheradresse (wie die Adresse des Labels

`zahl`) passt nicht in eine einzelne Instruktion, da auch Platz für den Opcode und die Register benötigt wird. Die Adresse wird daher in zwei Schritten berechnet: `auipc` berechnet die Basisadresse der Seite relativ zum Program Counter (PC), und der darauffolgende `lh`-Befehl lädt den Wert mit dem verbleibenden Offset.

2. `li t1, 0`

- *Übersetzung:* `addi x6, x0, 0`
- *Erklärung:* Es gibt in RISC-V keine dedizierte Hardware-Instruktion, um eine Konstante direkt in ein Register zu laden. Stattdessen nutzt man die vorhandene Additions-Instruktion und addiert die Konstante (hier 0) zum `zero`-Register (`x0`), das immer den Wert 0 hat. Das Ergebnis landet im Zielregister `t1`.

3. `nop`

- *Übersetzung:* `addi x0, x0, 0`
- *Erklärung:* Der Prozessor benötigt keinen speziellen Schaltkreis für „Tue nichts“. Um einen Taktzyklus zu verbrauchen, ohne den Zustand des Prozessors zu ändern, führt man eine Operation ohne Seiteneffekte aus: Man addiert 0 zu 0 und schreibt das Ergebnis in das Register `x0`. Da `x0` schreibgeschützt ist (immer 0 bleibt), hat dies keinen Effekt, außer dass Zeit vergeht.

1.3

Was machen die einzelnen Segmente dieses Programmes? Welche Bedeutung haben die jeweiligen Register dabei? Welche Funktion wird durch das Programm berechnet?

Lösung:

`.data`: Der symbolische Name `zahl` wird deklariert und initialisiert.
`.text`: Register werden vorgeladen, eine Addition wird in einer Schleife ausgeführt, bis eine Schlussbedingung erreicht wird.

Funktion der Register:

`t0` wird mit dem Input geladen und ist der Counter für unsere Schleife.
`t1` enthält die Konstante 0 und dient dazu, die Schleife zu beenden.
`t2` speichert das Ergebnis der Addition.

Funktion des Programms:

Sei h die Funktion, die die unteren 16 Bits einer binären Zahl ausgibt (half

word). Dann ist unser Programm die Funktion

$$f(x) = \sum_{n=1}^{\lfloor h(x)/16 \rfloor} n$$

mit $x = \text{zahl}$. Umgangssprachlich formuliert, summiert das Programm die natürlichen Zahlen bis zu einer gewissen Zahl.

1.4

Das Programm erhält einen Wert als Eingabe und gibt einen Wert als Ausgabe zurück. Stehen Ein- und Ausgabe in Registern oder im Speicher? An welcher Stelle?

Lösung:

Die Eingabe des Programms ist der symbolische Name **zahl**, der im **.data**-Segment deklariert wird. Dies geschieht im Speicher. Diese primäre Eingabe wird im Segment **beginning** ins Register **t0** geladen und modifiziert. Der Inhalt von **t0** am Ende von **beginning** ist die Ausgangsposition beim Start der Additions-Schleife – man könnte diesen State auch als Input sehen.

Das Register **t2** interpretieren wir als Ausgabe.

1.5. RISC-V benutzt little-endian. Würde sich das Ergebnis ändern, wenn eine andere Endianess benutzt werden würde?

Ja, das Ergebnis würde sich ändern.

Begründung: Das Datensegment enthält das Wort **0x09fa00ce**. Die Reihenfolge, in der die Bytes dieses Wortes im Speicher abgelegt werden, hängt von der Byte-Reihenfolge (Endianness) ab.

- **Little-Endian (RISC-V):** Das niederwertigste Byte wird an der niedrigsten Adresse gespeichert.
 - Speicherinhalt (Adresse x bis $x + 3$): **CE, 00, FA, 09**.
 - Der Befehl **lh t0, zahl** lädt die zwei Bytes an der niedrigsten Adresse (CE und 00). Das System interpretiert sie als **0x00CE** (206_{10}).
- **Big-Endian:** Das höchstwertige Byte wird an der niedrigsten Adresse gespeichert.
 - Speicherinhalt (Adresse x bis $x + 3$): **09, FA, 00, CE**.

- Der Befehl `lh t0, zahl` würde in diesem Fall die Bytes 09 und FA laden. Das System würde sie als `0x09FA` (2554_{10}) interpretieren.

Da der geladene Wert in `t0` als Startwert für die Berechnung dient, würde ein anderer Wert (2554 statt 206) zu einer völlig anderen Anzahl von Schleifendurchläufen und somit zu einem anderen Endergebnis führen.

1.6. Schreiben Sie das Programm so um, dass weniger Instruktionen und temporäre Register benötigt werden.

Man kann das Register `t1` und die Instruktion `li t1, 0` einsparen, indem man für den Vergleich direkt das `zero`-Register (`x0`) verwendet, das in RISC-V immer den Wert 0 hat.

1 .data		
2 zahl: .word 0x09fa00ce		
3		
4 .text		
5 beginning:		
6 lh t0 zahl		
7 srli t0 t0 4		
8 #li t1 0 #reduzante Instruktion		
9 middle:		
10 add t2 t2 t0		
11 addi t0 t0 -1		
12 bne t0 x0 middle #Vergleich direkt mit x0		
13 nop		

00000000 <beginning>:		
0: 10000297	auipc x5 0x10000	
4: 00029283	lh x5 0 x5	
8: 0042d293	srli x5 x5 4	
0000000c <middle>:		
c: 005383b3	add x7 x7 x5	
10: fff28293	addi x5 x5 -1	
14: fe029ce3	bne x5 x0 -8 <middle>	
18: 00000013	addi x0 x0 0	

Abbildung 2: Optimierter Code in Ripes

Einsparung:

- **Instruktionen:** 1 weniger (die Pseudoinstruktion `li` bzw. die echte Instruktion `addi` entfällt).
- **Register:** 1 weniger (das temporäre Register `t1` wird nicht mehr benötigt).

Aufgabe 2 *RISC-V*

2.2. Analyse der `auipc`-Instruktion

1. **Warum braucht `auipc` mehr Bits?** RISC-V-Instruktionen sind 32 Bit lang, ebenso wie die Speicheradressen. Da in einer Instruktion auch Platz für den Opcode und Zielregister benötigt wird, kann eine volle 32-Bit-Adresse nicht in einer einzigen Instruktion untergebracht werden. Die Architektur teilt die Adresse daher auf:

$$32 \text{ Bit} = 20 \text{ Bit (Upper Immediate)} + 12 \text{ Bit (Lower Immediate)}$$

Die `auipc`-Instruktion (U-Type) übernimmt die oberen 20 Bits, während nachfolgende I-Type-Instruktionen (z.B. `addi` oder `load`) die restlichen 12 Bits liefern. Zusammen decken sie den gesamten Adressraum ab.

2. **Was würde sich ändern bei nur 12 Bit?** Hätte `auipc` nur 12 Bit, könnten wir in Kombination mit einer weiteren 12-Bit-Instruktion maximal 24 Bit einer Adresse konstruieren ($12 + 12 = 24$). Der erreichbare Adressraum würde von 2^{32} (4 GB) auf 2^{24} (16 MB) schrumpfen. Das Laden von Adressen, die weiter entfernt liegen, wäre mit dem Standard-Mechanismus nicht mehr möglich.
3. **Gegenmaßnahmen:** Um dennoch eine volle 32-Bit-Adresse zu konstruieren, wären deutlich mehr Instruktionen nötig. Man müsste den Wert stückweise laden und verschieben (Shift-Operationen), zum Beispiel:
 - (a) Lade Teil 1.
 - (b) Schiebe nach links (`slli`).
 - (c) Addiere Teil 2.
 - (d) Schiebe nach links.
 - (e) ...

Dies würde die Codegröße erhöhen und die Performance verringern.

2.3 Malte hat ein Assembly-Programm geschrieben, um sich mit der Speicherstruktur von RISC-V vertraut zu machen. Leider ist seine Katze über die Tastatur gelaufen und hat die Zeile 8 verunstaltet. Er weiß noch, dass er entweder ein Byte, Half word oder Word von der Adresse in `t0` in `t1` geladen hat und die Ausgabe in `t1` am Ende `0xfeed0ca7` war.

```
1 .data
2 zahl1: .word 0x0ca7be5a
3 zahl2: .word 0xbaedfeed
4
5 .text
6 la t0 zahl1
7 addi t0 t0 2
8 pmbokibvcy7sxz6rewq
```

2.3 a) Welche Ausgaben erhalten Sie jeweils für das Laden eines Bytes, Halb- wortes und Wortes? Wie ändert sich das Ergebnis im Allgemeinen für ein unsigned Laden?

Analyse des Speicherinhalts (Little-Endian): Die Werte 0x0ca7be5a (zahl1) und 0xbaedfeed (zahl2) liegen im Speicher. Da RISC-V *Little-Endian* verwendet, steht das niedrigwertigste Byte (LSB) an der niedrigsten Adresse. Angenommen **zahl1** liegt an Adresse 0x00:

Offset	Byte (Hex)	Herkunft
+0	5A	LSB von zahl1
+1	BE	
+2	A7	$\leftarrow$ t0 nach <code>addi t0, t0, 2</code>
+3	0C	MSB von zahl1
+4	ED	LSB von zahl2
+5	FE	

Ergebnisse der Ladebefehle (Offset +2): Der Zeiger t0 zeigt auf das Byte 0xA7.

- **Byte laden (1b):** Geladen wird 0xA7 (10100111₂). Da das oberste Bit 1 ist, führt 1b (signed load) eine Vorzeichenerweiterung (Sign-Extension) durch.

Ergebnis: 0xFFFFFA7

- **Halbwort laden (1h):** Es werden 2 Bytes geladen: 0xA7 (Adresse +2) und 0x0C (Adresse +3). Little-Endian setzt diese zu 0x0CA7 zusammen. Das oberste Bit (0) ist positiv.

Ergebnis: 0x00000CA7

- **Wort laden (1w):** Es werden 4 Bytes geladen: A7, 0C, ED, FE. Zusammengesetzt ergibt dies 0xFEED0CA7. Dies entspricht Maltes Beobachtung.

Ergebnis: 0xFEED0CA7

Unterschied bei unsigned Laden:

Bei Instruktionen wie 1bu (Load Byte Unsigned) findet keine Vorzeichenerweiterung statt; stattdessen werden die oberen Bits mit Nullen aufgefüllt. Für das Byte 0xA7 wäre das Ergebnis 0x000000A7 (statt 0xFF...). Bei Halfword/Word ändert sich hier nichts, da das MSB bereits 0 bzw. relevant für den Wert ist.

2.3 b) Verändertes Alignment (+3)

Wenn die Instruktion zu `addi t0, t0, 3` geändert wird, zeigt t0 auf eine ungerade Adresse (Offset +3).

- **Byte:** Zugriff ist immer aligned (1-Byte-Ausrichtung).
- **Halfword:** Benötigt eine durch 2 teilbare Adresse. 3 ist ungerade → **Misaligned**.
- **Word:** Benötigt eine durch 4 teilbare Adresse. 3 ist nicht durch 4 teilbar → **Misaligned**.

2.3 c) Zugriff auf nicht ausgerichteten Speicher

Die CPU kann bei ungültigem Alignment durch wiederholtes Speichern und Laden konteragieren. Die genauen Techniken hängen von der Implementation ab (shifting, masking, merging). Solche Operationen sind komplexer, langsamer (wiederholtes Lesen == mehr Zyklen) und evtl. nicht mehr atomar – es wird nicht mehr garantiert, dass sie 'in einem Schritt' geschehen.

Aus der RISC-V Spezifikation:

"Loads and stores whose effective address is not naturally aligned to the referenced datatype (i.e., the effective address is not divisible by the size of the access in bytes) have behavior dependent on the EEI.

(...)

Even when misaligned loads and stores complete successfully, these accesses might run extremely slowly depending on the implementation (e.g., when implemented via an invisible trap). Furthermore, whereas naturally aligned loads and stores are guaranteed to execute atomically, misaligned loads and stores might not, and hence require additional synchronization to ensure atomicity."

Quelle: "<https://docs.riscv.org/reference/isa/unpriv/rv32.html#ldst>"

Aufgabe 3 *Load-Store Architektur*

1. Äquivalente RISC-V Programme

Annahme: Die Adressen A, B und C liegen bereits in den Registern x10, x11 und x12 vor. Wir gehen davon aus, dass es sich bei den Daten um 32-Bit-Wörter handelt. Daher verwenden wir lw/sw und erhöhen Zeiger um 4 Bytes.

a) MUL [A], [B], [C] In einer Load-Store-Architektur müssen Operanden erst in Register geladen werden.

```

lw t0, 0(x11)    # Lade Wert von Adresse B in t0
lw t1, 0(x12)    # Lade Wert von Adresse C in t1
mul t0, t0, t1    # Multipliziere: t0 = t0 * t1
sw t0, 0(x10)    # Speichere Ergebnis an Adresse A

```

b) MOV [A], [B] Ein direkter Speichertransfer ist nicht möglich. Ein temporäres Register wird benötigt.

```

lw t0, 0(x11)    # Lade Wert von Adresse B in t0
sw t0, 0(x10)    # Speichere Wert aus t0 an Adresse A

```

c) REDUCE_SUM [A], [B], [C] Dies erfordert eine Schleife. Wir summieren Werte beginnend bei Adresse B bis zur Adresse C.

```

        li t0, 0          # t0 ist der Akkumulator (Summe), init mit 0
        mv t1, x11        # t1 ist der temporäre Pointer, startend bei B
loop:
    bgtu t1, x12, end      # Abbruchbedingung: Wenn Pointer > Adresse C
    lw t2, 0(t1)          # Lade Wert vom aktuellen Pointer
    add t0, t0, t2         # Addiere Wert zur Summe
    addi t1, t1, 4         # Pointer erhöhen (+4 Bytes für 32-Bit Word)
    j loop                # Sprung zurück zum Anfang
end:
    sw t0, 0(x10)         # Speichere Gesamtsumme an Adresse A

```

2. Vor- und Nachteile der Load-Store Architektur

Eine Load-Store-Architektur hat die Konsequenz, dass wir Speicheroperationen immer explizit durchführen ('load' und 'store') – es werden mehr Register und Instruktionen gebraucht. Der Vorteil ist, dass unser instruction set vereinfacht wird. Arithmetische Operationen benutzen ausschließlich Register. Das vereinheitlicht das Instruktionsformat und erleichtert dadurch die Dekodierung.

3. Architektonische Änderungen für MOV [A], [B]

Eine Load-Store-Architektur erlaubt höchstens einen Speicheroperanden pro Instruktion. Die Instruktion 'MOV [A], [B]' besitzt zwei Speicheroperanden. Damit müsste eine einzelne Instruktion sowohl aus dem Speicher lesen und in den Speicher schreiben. Das Instruktionsformat, die Dekodierung und die Hardware würden mitunter komplizierter ausfallen.

Wir müssten mindestens die folgenden Fragen beantworten:

- wie werden die Speicheroperationen miteinander koordiniert?

- wie modifizieren wir die Speicherarchitektur, damit ein dualer Zugriff im selben Takt möglich wird (Zusatzport)?
- wie passen wir das Instruktionsformat so an, dass wir das entsprechende Operandenmuster signalisieren können (beide sind Adressen)?
- wie gehen wir mit zwei 32-Bit Speicheradressen um in einem Schema mit 32-Bit Instruktionen?

Aufgabe 4 *Pulsbreitenmodulation*

```
.data
white:                .word 0x00ffffff
black:                .word 0x0

pulseLength:         .word 100
inputSpeed:           .word 20
initialBrightness:    .word 40

# --- REGISTERS ---
# s0: current brightness
# s1: ticks left in current period
# s2: ticks before OFF
# s3: ticks before input
# s4: color

.text
start:
    lw s0, initialBrightness
    lw s1, pulseLength
    add s2, s0, x0        # set ON counter to initial brightness
    lw s3, inputSpeed

tick:
    addi s1, s1, -1
    beqz s1, drawPixel    # start new period

    addi s2, s2, -1        # when ON ends -> set to OFF
    beqz s2, removePixel

    addi s3, s3, -1        # handle input periodically
    beqz s3, checkInput

    j tick

checkInput:
    lw s3, inputSpeed     # reset input counter
```

```

        li t0, D_PAD_0_UP          # if DPAD UP
        lb t1, 0(t0)
        bnez t1, increaseBrightnessTestBounds

        li t0, D_PAD_0_DOWN        # if DPAD DOWN
        lb t1, 0(t0)
        bnez t1, decreaseBrightnessTestBounds

        j tick                      # else

increaseBrightnessTestBounds:
        lw t0, pulseLength          # if current_brightness < pulselength-1
        addi t0, t0, -1
        blt s0, t0, increaseBrightness
        j tick                      # else

increaseBrightness:
        addi s0, s0, 1
        j tick

decreaseBrightnessTestBounds:
        bgtz s0, decreaseBrightness # if current_brightness > 0
        j tick                      # else

decreaseBrightness:
        addi s0, s0, -1
        j tick

drawPixel:
        lw s1, pulseLength          # reset tick counter
        add s2, s0, x0              # reset ON counter

        beqz s2, removePixel        # if brightness==0 set to OFF

        lw s4, white                # set color for painting
        j paintCanvas

removePixel:
        lw s4, black
        j paintCanvas

paintCanvas:
        li t1, LED_MATRIX_0_BASE
        sw s4, 0(t1)
        j tick

```